Comparative Evaluation and Targeted Fine-Tuning of Pretrained CNN Architectures for Chest X-ray Pneumonia Classification

Taha Ebaad
Image Analysis Lab, SINES
National University of Sciences and Technology (NUST)
Islamabad, Pakistan
Intern

***Abstract*—This paper presents a two-part experimental study of pneumonia classification on the Chest X-Ray Pneumonia dataset (4,433 training, 783 validation, 624 test images) using ImageNet-pretrained convolutional neural network (CNN) backbones. First, a comparative evaluation of eleven pretrained backbones is conducted under a common, frozen-feature-extraction pipeline, split across two independent notebook runs; a forensic code review identified that four of eleven architecture sections in the first run did not correctly instantiate their labeled backbone, and these results are excluded from the reproducible seven-architecture baseline reported here rather than retained under an incorrect label. Among the seven verified baseline architectures, InceptionResNetV2 achieved the highest test accuracy (0.8077) and was selected for a subsequent targeted fine-tuning experiment. In this second stage, InceptionResNetV2 was fine-tuned in two stages – an initial frozen-backbone stage followed by partial unfreezing of its final 80 layers – and re-evaluated using a compact, globally-pooled 1,536-dimensional feature representation. This fine-tuned configuration achieved a test accuracy of 0.8606, a weighted F1 score of 0.8523, and a Cohen's kappa of 0.6799, substantially outperforming the same architecture's frozen, untrained-head comparative configuration (accuracy 0.6875, F1 0.6214, κ 0.2145). This controlled, same-backbone comparison – rather than any comparison across different architectures – is presented as the central evidence for the value of targeted fine-tuning in this study. The eleven-architecture comparative evaluation is explicitly distinguished throughout from this fine-tuning experiment: none of the eleven backbones in the comparative run were fine-tuned, and the paper does not claim otherwise. No confusion matrix is presented, since exact prediction arrays were not available for reconstruction; per-class precision, recall, and F1 scores are instead reported verbatim from the notebooks' own classification-report output. No claim of state-of-the-art performance is made.**

***Index Terms*—pneumonia classification, chest X-ray, convolutional neural networks, transfer learning, fine-tuning, deep feature extraction, InceptionResNetV2**

I. Introduction

Pneumonia remains a significant cause of morbidity in pediatric populations, and chest radiography is a standard, widely available diagnostic tool for its detection. Manual interpretation of chest X-rays, however, is time-consuming and subject to inter-observer variability, motivating substantial interest in automated classification systems built on convolutional neural networks (CNNs). A common approach in this literature is transfer learning, in which an ImageNet-pretrained CNN backbone is repurposed as a feature extractor or lightly retrained for a binary NORMAL/PNEUMONIA classification task [1]–[3].

This study investigates pneumonia classification on the Chest X-Ray Pneumonia dataset using two complementary experimental designs, implemented across two Jupyter notebooks over the course of an internship project. The first is a comparative evaluation across eleven ImageNet-pretrained CNN backbones under a common, untrained feature-extraction pipeline, intended to establish which architecture produces the strongest feature representation for this task without the cost of fine-tuning every candidate backbone. The second is a targeted fine-tuning experiment, in which the single strongest architecture identified by the comparative evaluation – InceptionResNetV2 – is fine-tuned in two stages and re-evaluated using a compact, globally-pooled feature representation.

During the preparation of this paper, a forensic code review of both notebooks revealed that four of the eleven architecture sections in the first (unoptimized) notebook did not correctly instantiate their labeled backbone, due to a copy-paste implementation error; these four sections' results are excluded from the reproducible baseline reported in Section VA rather than silently corrected or retained under an incorrect label. This exclusion, along with several other reproducibility considerations uncovered during the same review, is documented transparently in Section VII rather than omitted from the paper, in keeping with the view that an honest account of what was actually executed is more valuable than a cleaner-looking but inaccurate one.

The central contribution of this paper is not a claim of state-of-the-art pneumonia-classification performance, but a controlled, same-backbone comparison: InceptionResNetV2 evaluated first as a frozen, untrained-head feature extractor (accuracy 0.6875 in the optimized-notebook comparative run) and then as a genuinely fine-tuned model with a globally-pooled 1,536-dimensional feature representation (accuracy 0.8606). This paired comparison isolates the effect of fine-tuning on a single, fixed architecture, holding the backbone constant while the training regime changes – a comparison this paper argues

is more informative than treating the entire eleven-architecture comparative run as though every backbone had been fine-tuned, which it was not.

The remainder of this paper is organized as follows. Section II reviews related literature on CNN-based pneumonia detection, transfer learning, fine-tuning, and deep feature extraction. Section III describes the dataset, its actual observed distribution, and the two preprocessing regimes used in this study. Section IV details the baseline architectures, the common comparative feature-extraction pipeline, the downstream classifiers, and the targeted fine-tuning procedure. Section V reports the reproducible baseline results, the eleven-architecture comparative results, the fine-tuning results, per-class performance, and the central frozen-versus-fine-tuned comparison. Section VI interprets these results. Section VII documents the reproducibility issues uncovered during this study's preparation. Section VIII concludes.

II. Related Work

A. CNN-based Pneumonia Detection

Convolutional neural networks applied to chest radiographs have become a common approach for automated pneumonia screening, typically formulated as a binary classification task distinguishing NORMAL from PNEUMONIA images [1]–[3]. Reported studies vary considerably in dataset size and composition: the Kaggle Chest X-Ray Images (Pneumonia) dataset in its most commonly cited form contains 5,863 pediatric images [2], while other studies use related but distinct pediatric collections of similar size [1], [3], and at least one uses a substantially larger adult-population dataset drawn from the RSNA Pneumonia Detection Challenge (26,684 images) [4]. These dataset differences are material to any comparison of reported accuracy figures across studies and are treated explicitly rather than glossed over in Section VI.

B. Transfer Learning

ImageNet-pretrained CNN backbones – most commonly VGG16, VGG19, ResNet50, and MobileNetV2 – are widely reused as fixed or lightly adapted feature extractors for pneumonia classification [1], [2], [5]. Wibowo et al. [2] directly compare a custom CNN, ResNet50, and VGG16 under transfer learning on the same 5,863-image Kaggle dataset referenced above, reporting ResNet50 as the strongest of the three. Pal and Pal [5] similarly compare a custom CNN against VGG16 and MobileNetV2 transfer learning before combining a custom CNN with MobileNetV2 in a hybrid architecture, finding that the hybrid model matched but did not exceed VGG16's standalone accuracy – an outcome the authors note explicitly rather than treating as an unqualified success, and one that parallels a comparable finding in the present study (Section VE).

C. Fine-Tuning

A smaller number of studies distinguish explicitly between a frozen-backbone transfer-learning regime and a genuine fine-tuning regime in which backbone weights are updated. Chedsom [1] compares unmodified VGG16/VGG19 transfer learning against the same backbones equipped with a deeper, trained fully-connected head, reporting a measurable improvement from the customized configuration. Choudhury [3] provides the closest methodological analogue to the present study's central experiment, directly comparing frozen-backbone and fine-tuned configurations of ResNet50, DenseNet121, and EfficientNet-B0 on a pediatric pneumonia dataset and reporting that fine-tuning outperformed the frozen-backbone configuration by an average of 5.5 percentage points. This frozen-versus-fine-tuned framing directly motivates the targeted InceptionResNetV2 fine-tuning experiment described in Section IVD and the corresponding same-backbone comparison in Section VE, though it is noted that the present study's magnitude of improvement and absolute performance are not directly comparable to Choudhury's, given the differences in dataset and preprocessing discussed in Section VI.

D. Deep Feature Extraction and Downstream Classification

A related line of work extracts fixed-length feature vectors from a pretrained or task-adapted CNN and classifies them with a separate downstream model rather than an end-to-end trained classification head. Satish Chandra et al. [4] extract deep features from the fully-connected layers of ResNet50, VGG16, and DenseNet121 and classify them with NGBoost and several classical baselines, reporting their strongest configuration (ResNet50 features with NGBoost) at 0.98 accuracy on the RSNA dataset. This deep-features-plus-classifier pattern structurally parallels the comparative feature-extraction pipeline used throughout Section IVB–C of the present study, in which a common Flatten-based feature vector is classified with both a feed-forward network and a set of classical classifiers (KNN, SVM, random forest, AdaBoost, and XGBoost). A key methodological difference is noted: the features used by Satish Chandra et al. are drawn from a network whose fully-connected layers were themselves trained for a related classification task, whereas the comparative pipeline's 64-dimensional feature vector in the present study is produced by an untrained, randomly initialized head, a distinction that is discussed further in Section VII.

Across the five reviewed studies, accuracy, precision, recall, and F1 score are reported consistently, with area under the ROC curve or specificity reported in several [2]–[4]; none of the five reviewed studies report Cohen's kappa, despite class imbalance being discussed explicitly in at least two of them [2], [4]. The present study reports Cohen's kappa alongside the more conventional metrics throughout Section V for this reason.

III. Dataset and Preprocessing

A. Dataset

This study uses the Chest X-Ray Pneumonia dataset, a publicly available collection of pediatric chest radiographs commonly used in the pneumonia classification literature and cited by several of the reviewed papers (e.g., [2]). Each image is labeled as either NORMAL or PNEUMONIA, and the

TABLE I: Dataset Distribution

Split	Images	% of Total
Training	4,433	75.9%
Validation	783	13.4%
Test	624	10.7%
Total	**5,840**	**100.0%**

classification task is treated as binary throughout this study. The publicly distributed release of this dataset is commonly reported to contain 5,863 images in total; the experiments in this paper, however, loaded and processed 5,840 images, a discrepancy discussed further below.

B. Data Distribution

Table I reports the exact split sizes used in every experiment described in this paper. The gap between the commonly cited 5,863-image total and the 5,840 images actually loaded and processed is only partially explained by the implementation: the dataset's shipped validation folder was found to contain only 32 images at load time, below a minimum-validation-size threshold of 200 images built into the data loader, which triggered the loader to instead carve a validation split out of the training folder. This mechanism accounts for the re-partitioning of the data but does not, by itself, fully account for the exact 23-image difference between 5,863 and 5,840; a possible additional contributor is a small number of images that failed to decode during loading, but this was not separately logged in the executed pipeline and is not claimed as a confirmed explanation. The 5,840-image count is reported here as the authoritative, directly observed figure for this study, rather than the commonly cited 5,863.

The training-set class balance was not printed directly by the data loader. It can be estimated from the balanced class weights computed for the fine-tuning experiment (Section IVD), which were {NORMAL: 1.944, PNEUMONIA: 0.673}; inverting this balanced-weighting ratio against the known training-set size of 4,433 images yields an estimated class distribution of approximately 1,140 NORMAL and 3,293 PNEUMONIA images (roughly 26% / 74%). This figure is reported as a derived estimate rather than a directly measured count, and the resulting class imbalance motivated the use of weighted precision/recall/F1 averaging and class-weighted training in the fine-tuning experiment, both described in Section IV.

C. Train/Validation/Test Split

All experiments in this paper use the same fixed split of 4,433 training, 783 validation, and 624 test images (Table I), loaded once and reused identically across every architecture and both experimental pipelines described in Section IV. No additional cross-validation was performed.

D. Image Preprocessing

Two distinct preprocessing regimes were used in this study, corresponding to its two experimental components. In the comparative evaluation pipeline (Section IVA–C), images were decoded and resized to $224 \times 224 \times 3$ using OpenCV's `cv2.imread`, which returns a three-channel BGR array directly; the resulting raw uint8 values were passed to each pretrained backbone without channel reordering, mean subtraction, or scaling. This lack of architecture-specific normalization in the comparative pipeline is a known characteristic of the implementation, not an omission introduced during the writing of this paper, and is discussed further as a limitation in Section VII.

In the targeted fine-tuning pipeline (Section IVD), images were explicitly converted from BGR to RGB channel order and passed through the architecture-specific `preprocess_input` function associated with InceptionResNetV2, which was confirmed to rescale pixel values to the range $[-1, 1]$. This preprocessing difference between the two pipelines is intentional and is treated as part of the fine-tuning procedure rather than a confound to be normalized away, since correcting this known limitation of the naive comparative pipeline was one of the stated motivations for the targeted fine-tuning experiment.

TABLE II: Backbone Configuration Summary (Common Comparative Pipeline)

Architecture	Weights	include_top	Input	Feature dim.
ResNet50	ImageNet	False	224×224×3	64
VGG16	ImageNet	False	224×224×3	64
VGG19	ImageNet	False	224×224×3	64
ResNet101	ImageNet	False	224×224×3	64
MobileNetV2	ImageNet	False	224×224×3	64
MobileNet	ImageNet	False	224×224×3	64
InceptionV3	ImageNet	False	224×224×3	64
InceptionResNetV2	ImageNet	False	224×224×3	64
DenseNet169	ImageNet	False	224×224×3	64
DenseNet121	ImageNet	False	224×224×3	64
XceptionNet	ImageNet	False	224×224×3	64

IV. Methodology

A. Baseline CNN Architectures

Eleven ImageNet-pretrained CNN architectures were nominally included in the comparative design of this study: ResNet50, VGG16, VGG19, ResNet101, MobileNetV2, MobileNet, InceptionV3, InceptionResNetV2, DenseNet169, DenseNet121, and Xception. Each architecture was instantiated with `include_top=False` and ImageNet-pretrained weights, exposing its convolutional feature maps for the common feature-extraction head described in Section IVB. Table II summarizes this configuration for all eleven architectures.

Not every architecture produced a genuinely traceable result in both notebooks used for this study. In the unoptimized notebook, code inspection confirmed that the sections labeled InceptionV3, DenseNet169, DenseNet121, and XceptionNet did not instantiate their labeled backbone: the section labeled InceptionV3 in fact instantiated MobileNet, and the sections labeled DenseNet169, DenseNet121, and XceptionNet all instantiated InceptionResNetV2. An additional implementation

issue compounded this for XceptionNet, whose result variables overwrote those of the preceding DenseNet121 section. These four architectures are therefore excluded from the unoptimized baseline results reported in Section VA; only the seven architectures whose executed code was verified to match their label – ResNet50, VGG16, VGG19, ResNet101, MobileNetV2, MobileNet, and InceptionResNetV2 – are reported as the reproducible unoptimized baseline. This exclusion, and its rationale, is discussed further in Section VII.

A second, independent run of the same eleven-architecture comparison was carried out in the optimized notebook. Code inspection confirmed that all eleven sections in this second run instantiate the backbone matching their label, and that each section uses a distinct set of result variables with no overwriting. All eleven architectures are therefore reported for this second comparative run (Section VB). This second run is referred to throughout this paper as the *optimized-notebook comparative evaluation* to distinguish it clearly from the targeted fine-tuning experiment described in Section IVD; the term "optimized" here refers only to the notebook file in which this evaluation was carried out, and does not imply that any of the eleven architectures in this comparison underwent fine-tuning.

B. Common Deep Feature Extraction Pipeline

Both the unoptimized baseline and the optimized-notebook comparative evaluation use an identical feature-extraction and classification head applied on top of each backbone's raw convolutional output:

- Flatten
- Dense(64, kernel_initializer=`he_uniform`)
- BatchNormalization – active only in the section literally named "ResNet50" in both notebooks; commented out in every other section
- ReLU
- Dense(128) → ReLU
- Dense(256) → ReLU
- Dense(128) → ReLU
- Dense(64) → ReLU

The output of the final Dense(64)+ReLU layer forms a 64-dimensional feature vector, confirmed directly from the printed shape of the extracted training features in every section of both notebooks (`train_features.shape: (4433, 64)`). No Dropout layer is active anywhere in this pipeline; all Dropout lines present in the source code are commented out. Input images are fed to this pipeline without architecture-specific normalization, as described in Section IIID.

C. Downstream Classification

The 64-dimensional feature vector produced by each backbone is classified in two ways. First, a small feed-forward network (Dense 32 → 64 → 128 → 256 → 128 → 2, with no BatchNormalization or Dropout active) is trained for 10 epochs with the Adam optimizer and categorical cross-entropy loss, with no class weighting and no learning-rate scheduling. Second, five classical classifiers are trained on the same feature vector using library-default or lightly customized hyperparameters: k-nearest neighbors ($k = 5$, ball-tree algorithm), a support vector classifier with default settings, a random forest (`max_depth=9`, entropy criterion), AdaBoost with default settings, and XGBoost with default settings. The feed-forward network's test-set performance is the figure reported in the results tables of Section VA–B; it was confirmed to be the primary evaluated classifier for the comparative pipeline in both notebooks.

TABLE III: Targeted InceptionResNetV2 Fine-Tuning Configuration

	Stage 1 (frozen)	**Stage 2 (partial unfreeze)**
Backbone state	Fully frozen	Last 80 layers unfrozen (BatchNorm layers kept frozen)
Optimizer	Adam	Adam
Learning rate	1×10^{-3}	1×10^{-5}
Max epochs	8	20
Stopping	Ran full budget	Early stopping, epoch 12
Best val. accuracy	0.9259	0.9451
Class weights	NORMAL: 1.944, PNEUMONIA: 0.673	
Label smoothing	0.05	
Precision	Mixed precision	
Augmentation	Rotation, shift, h-flip, zoom, brightness jitter	

D. Targeted InceptionResNetV2 Fine-Tuning

Following the baseline comparison (Section VA), InceptionResNetV2 was selected as the strongest verified architecture and was the subject of a separate, more extensive fine-tuning experiment. This experiment was applied to InceptionResNetV2 only; it was not repeated for any of the other ten architectures evaluated in Sections VA–B, and this scope limitation is treated as a defining characteristic of the experimental design rather than an oversight.

The fine-tuning procedure used a two-stage schedule, summarized in Table III. In Stage 1, the InceptionResNetV2 backbone was kept fully frozen while a randomly initialized classification head was trained for up to 8 epochs with the Adam optimizer at a learning rate of 1×10^{-3}; this stage ran its full epoch budget and reached a best validation accuracy of 0.9259. In Stage 2, the last 80 layers of the backbone were unfrozen (with BatchNormalization layers within the backbone kept frozen to preserve their running statistics) and training continued for up to 20 epochs at a reduced learning rate of 1×10^{-5}; this stage stopped early at epoch 12 via an early-stopping callback monitoring validation accuracy, reaching a best validation accuracy of 0.9451. Both stages used class weights of {NORMAL: 1.944, PNEUMONIA: 0.673} to address the class imbalance described in Section IIIB, label smoothing of 0.05, mixed-precision training, and an image augmentation policy consisting of small rotations, shifts, horizontal flips, zoom, and brightness jitter.

E. Global Average Pooling Feature Representation

Unlike the Flatten-based 64-dimensional feature vector used in the comparative pipeline (Section IVB), the fine-tuned InceptionResNetV2 backbone's feature representation is obtained via GlobalAveragePooling2D applied to its final convolutional output, yielding a 1,536-dimensional feature vector. This was confirmed directly from the printed shapes of the extracted feature arrays for all three splits (train: 4,433 $\times$ 1,536; validation: 783 $\times$ 1,536; test: 624 $\times$ 1,536). This 1,536-dimensional representation was subsequently classified in the same two ways as the comparative pipeline's 64-dimensional representation: a purpose-built optimized feed-forward network (Dense 512 $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 2, with BatchNormalization and Dropout active, trained with class weighting, label smoothing, and early stopping as summarized in Table III), and a set of classical classifiers with separately hand-chosen hyperparameters (SVM with $C = 10$ and an RBF kernel, random forest with 200 trees, k-nearest neighbors with $k = 7$ and distance weighting, XGBoost with 200 trees at a learning rate of 0.05 and depth 5, and AdaBoost with 100 estimators at a learning rate of 0.5). These classifier hyperparameters were hand-chosen rather than selected via grid search, a limitation discussed further in Section VII.

F. Evaluation Metrics

All models in this study are evaluated on the held-out 624-image test set using five metrics: accuracy, weighted precision, weighted recall, weighted F1 score, and Cohen's kappa. Precision, recall, and F1 are computed with class-support weighting (`average='weighted'`) rather than macro-averaging, which is appropriate given the class imbalance described in Section IIIB. Accuracy is defined as

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{1}$$

Weighted precision, recall, and F1 are computed per class and combined using each class's support as its weight:

$$\text{Weighted F1} = \sum_{c} \frac{n_c}{N} \cdot F1_c, \tag{2}$$

where n_c is the number of test samples in class c and N is the total number of test samples. Cohen's kappa is computed as

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \tag{3}$$

where p_o is the observed agreement (equivalent to accuracy for this binary task) and p_e is the agreement expected by chance given the marginal class distributions of the true and predicted labels. Unlike raw accuracy, κ explicitly discounts the portion of agreement attributable to chance, which is a meaningful correction on an imbalanced dataset such as this one, where a classifier that always predicts the majority class can achieve a relatively high accuracy without any genuine discriminative ability. All five metrics are reported for every architecture and experimental configuration in Section V.

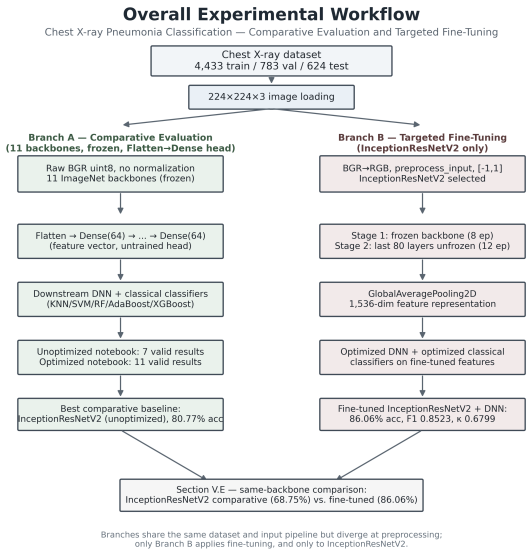

Fig. 1: Overall experimental workflow, showing the shared data pipeline, comparative evaluation of pretrained CNN backbones, and targeted InceptionResNetV2 fine-tuning branch.

TABLE IV: Reproducible Unoptimized Baseline Results

Model	Acc.	Prec.	Rec.	F1	κ
ResNet50	0.7308	0.7280	0.7308	0.7155	0.3835
VGG16	0.6266	0.5941	0.6266	0.5734	0.0925
VGG19	0.7067	0.7372	0.7067	0.6596	0.2802
ResNet101	0.7356	0.7405	0.7356	0.7143	0.3826
MobileNetV2	0.7436	0.7461	0.7436	0.7263	0.4074
MobileNet	0.6554	0.6402	0.6554	0.6110	0.1691
InceptionResNetV2	**0.8077**	**0.8183**	**0.8077**	**0.7968**	**0.5596**

V. Experimental Results

Figure 1 summarizes the overall experimental workflow described in Section IV. A shared data-loading and preprocessing pipeline feeds two experimental branches: a comparative evaluation branch and a targeted fine-tuning branch.

A. Reproducible Baseline Architecture Comparison

Table IV reports the unoptimized-notebook baseline results for the seven architectures whose executed code was verified to match their labeled backbone.

Among these seven architectures, InceptionResNetV2 achieved the highest accuracy (0.8077), weighted F1 score (0.7968), and Cohen's kappa (0.5596). MobileNetV2 was the second-strongest architecture by F1 score (0.7263) and Cohen's kappa (0.4074). VGG16 achieved the lowest accuracy (0.6266), F1 score (0.5734), and Cohen's kappa (0.0925).

The two representative architectures illustrate the baseline implementation at different points in the comparative evaluation. ResNet50 provides a representative view of the common feature-extraction head, while InceptionResNetV2 represents

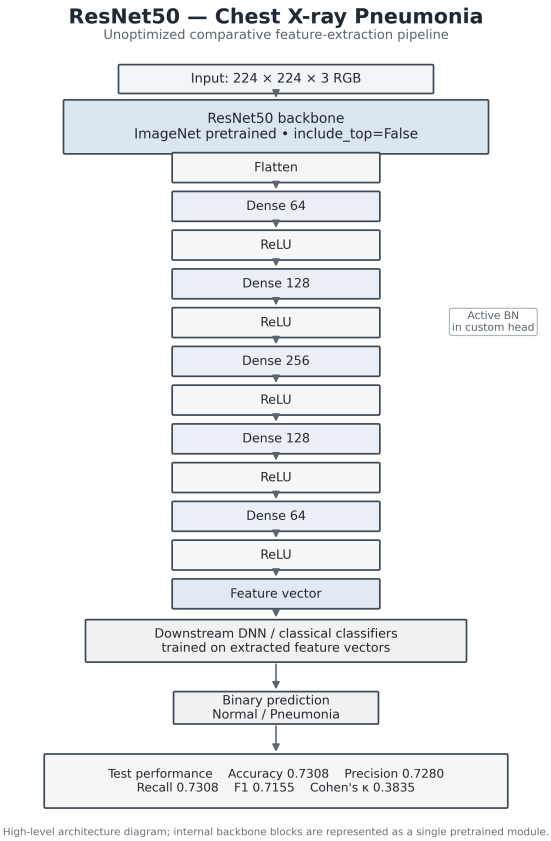

Fig. 2: ResNet50 architecture and feature-extraction/classification pipeline in the unoptimized baseline experiment.

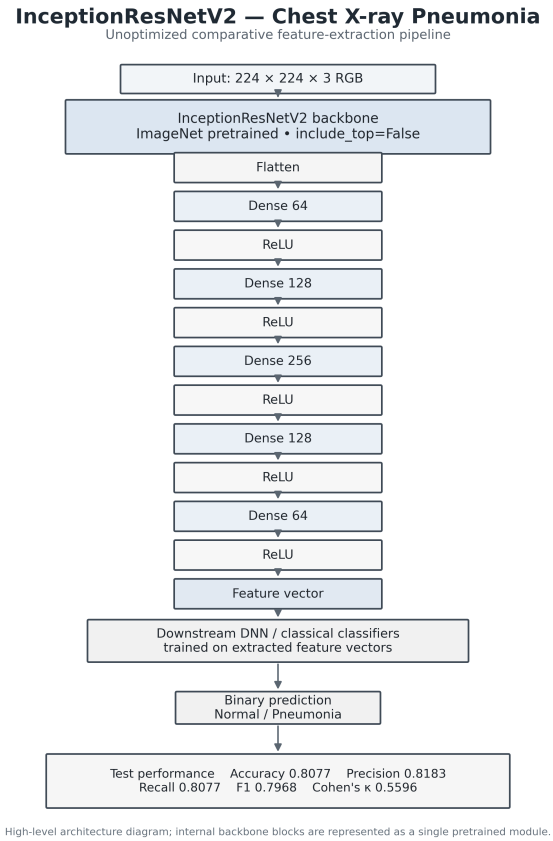

Fig. 3: InceptionResNetV2 architecture and feature-extraction/classification pipeline in the unoptimized baseline experiment.

the strongest verified baseline architecture according to the aggregate evaluation metrics.

Four architectures nominally included in the original comparative design—InceptionV3, DenseNet169, DenseNet121, and XceptionNet—are not reported in the reproducible baseline table because their corresponding sections in the unoptimized notebook did not instantiate the labeled backbone. Reporting those results under their intended labels would introduce an architecture-attribution error. They are therefore excluded from the authoritative baseline results.

B. Optimized Notebook Comparative Evaluation

A second, independent run of the eleven-architecture comparison was carried out in the optimized notebook. Table V reports the results for all eleven architectures whose labeled backbones were verified in this notebook.

InceptionV3 achieved the highest accuracy (0.8606), F1 score (0.8579), and Cohen's kappa (0.6934) in this comparative evaluation.

Two representative architectures from this second run are shown below.

The optimized-notebook comparative evaluation should not be interpreted as a fine-tuning experiment. The backbone architectures in this branch remain frozen, and the same general feature-extraction and downstream classification strategy is used across the eleven architectures.

In particular, the InceptionResNetV2 row in Table V, with accuracy 0.6875, F1 score 0.6214, and Cohen's kappa 0.2145, represents the frozen comparative configuration. It is therefore kept separate from the targeted fine-tuned InceptionResNetV2 experiment presented below.

TABLE V: Optimized-Notebook Comparative Evaluation Results (Frozen Backbones, Not Fine-Tuned)

Model	**Acc.**	**Prec.**	**Rec.**	**F1**	κ
ResNet50	0.7372	0.7430	0.7372	0.7158	0.3858
VGG16	0.6442	0.6552	0.6442	0.5478	0.0845
VGG19	0.6442	0.6275	0.6442	0.5777	0.1155
ResNet101	0.7083	0.7247	0.7083	0.6693	0.2946
MobileNetV2	0.7003	0.7420	0.7003	0.6449	0.2557
MobileNet	0.6715	0.6945	0.6715	0.6035	0.1759
InceptionV3	**0.8606**	**0.8613**	**0.8606**	**0.8579**	**0.6934**
InceptionResNetV2	0.6875	0.7358	0.6875	0.6214	0.2145
DenseNet169	0.7692	0.7898	0.7692	0.7480	0.4566
DenseNet121	0.7003	0.7176	0.7003	0.6565	0.2702
XceptionNet	0.7596	0.7685	0.7596	0.7418	0.4413

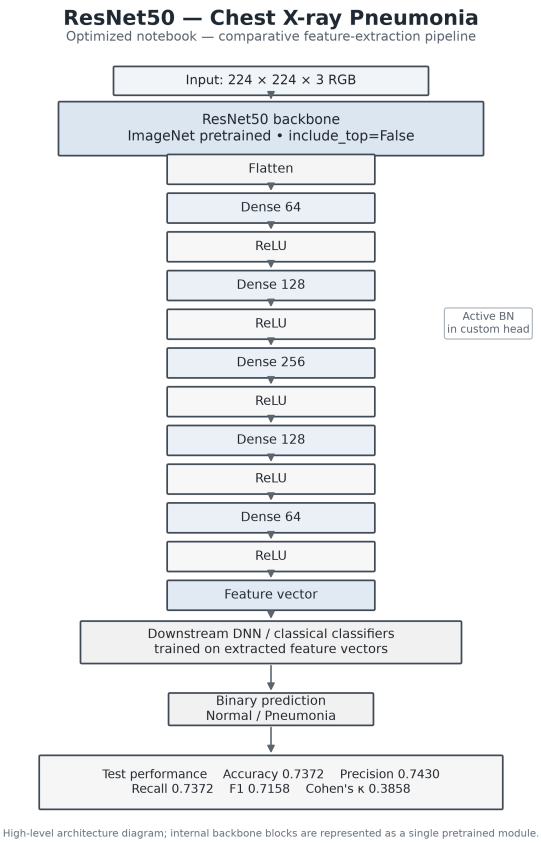

Fig. 4: ResNet50 architecture and comparative feature-extraction/classification pipeline in the optimized notebook. This configuration uses the frozen comparative pipeline and is not a fine-tuning experiment.

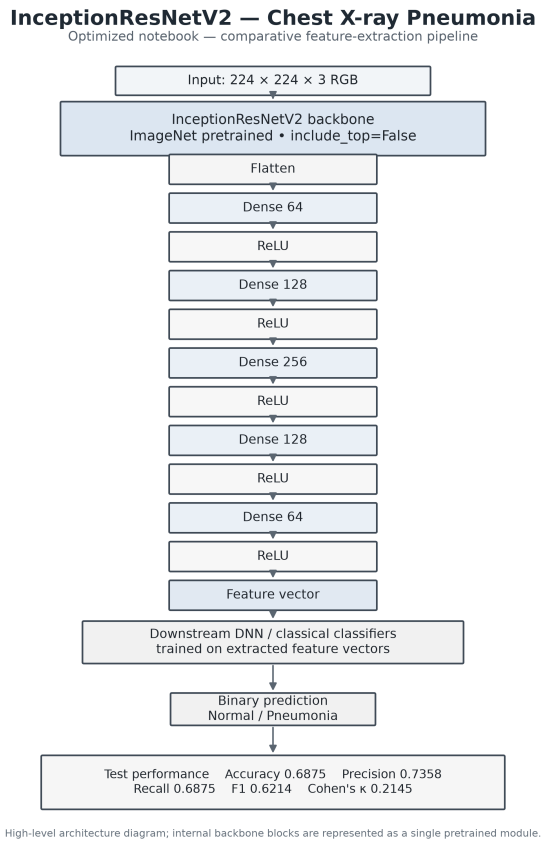

Fig. 5: InceptionResNetV2 architecture and comparative feature-extraction/classification pipeline in the optimized notebook. This configuration is frozen and is distinct from the subsequent fine-tuning experiment.

Similarly, the InceptionV3 accuracy of 0.8606 in this comparative table should not be interpreted as the fine-tuned InceptionResNetV2 result, despite the numerical coincidence between the two values.

C. Targeted InceptionResNetV2 Fine-Tuning

Following the comparative evaluation, InceptionResNetV2 was selected for a targeted fine-tuning experiment.

Unlike the preceding comparative experiments, this branch explicitly modified the training regime of the InceptionResNetV2 backbone through a two-stage fine-tuning strategy.

The fine-tuned InceptionResNetV2 model with the optimized feed-forward DNN achieved an accuracy of 0.8606, weighted F1 score of 0.8523, and Cohen's kappa of 0.6799.

For the same backbone, the frozen comparative configuration achieved an accuracy of 0.6875, weighted F1 score of 0.6214, and Cohen's kappa of 0.2145.

Thus, the same-backbone comparison provides the principal evidence for the effect of targeted fine-tuning in this study.

D. Per-Class Results

Table VII reports the per-class precision, recall, F1 score, and support directly from the notebook classification reports.

TABLE VI: Targeted InceptionResNetV2 Fine-Tuning: Classifier Comparison on the Fine-Tuned 1,536-Dimensional Feature Representation

Classifier	**Acc.**	**Prec.**	**Rec.**	**F1**	κ
Optimized DNN (reported)	**0.8606**	**0.8613**	**0.8606**	**0.8523**	**0.6799**
SVM (RBF, C=10)	0.8285	–	–	0.8139	0.5981
KNN (k=7, distance)	0.8189	–	–	0.8061	0.5803
XGBoost	0.8189	–	–	0.8035	0.5756
AdaBoost	0.8045	–	–	0.7860	0.5388
Random Forest	0.7772	–	–	0.7492	0.4638
For comparison — same backbone, frozen (not fine-tuned):					
InceptionResNetV2 (frozen)	0.6875	0.7358	0.6875	0.6214	0.2145

No confusion matrix is presented because exact prediction arrays were not available for direct reconstruction. Back-calculation of a confusion matrix from rounded classification-report values was deliberately avoided.

E. Baseline vs. Optimization Comparison

The primary optimization comparison holds the backbone architecture constant and compares the frozen and fine-tuned InceptionResNetV2 configurations.

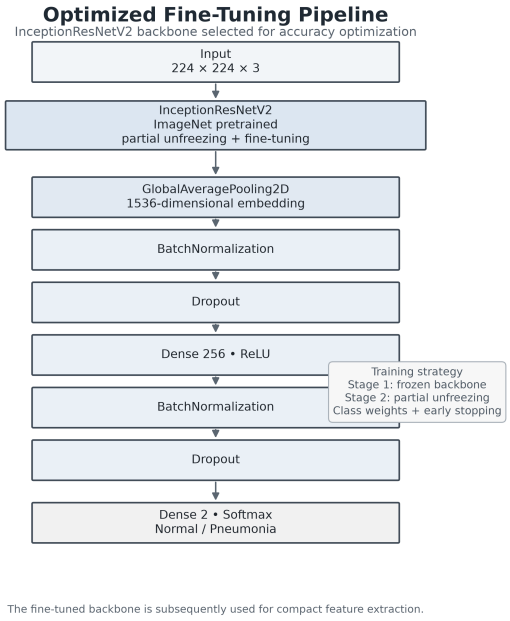

Fig. 6: Targeted InceptionResNetV2 fine-tuning pipeline, showing frozen Stage 1 training, partial backbone unfreezing during Stage 2, global average pooling to a 1,536-dimensional representation, and downstream classification.

TABLE VII: Per-Class Classification Report for InceptionResNetV2

Configuration	Class	Prec.	Rec.	F1	Support
Frozen comparative	NORMAL	0.87	0.57	0.69	234
	PNEUMONIA	0.79	0.95	0.86	390
Fine-tuned + DNN	NORMAL	0.99	0.64	0.77	234
	PNEUMONIA	0.82	0.99	0.90	390

Fine-tuning improved accuracy from 0.6875 to 0.8606, representing an increase of 0.1731, or 17.31 percentage points. Weighted F1 increased from 0.6214 to 0.8523, while Cohen's kappa increased from 0.2145 to 0.6799.

This same-backbone comparison represents the central optimization result of the study.

A separate paired comparison between the reproducible baseline and optimized-notebook comparative runs is also presented below. Importantly, this comparison does not represent deliberate fine-tuning; both configurations use the frozen comparative pipeline.

The paired comparison shows that ResNet50 improved across all three principal metrics, with accuracy increasing from 0.7308 to 0.7372, F1 from 0.7155 to 0.7158, and Cohen's kappa from 0.3835 to 0.3858.

However, these differences should not be interpreted as evidence of fine-tuning because the comparative pipeline itself was not deliberately modified between the two runs. The targeted InceptionResNetV2 fine-tuning experiment is therefore treated separately as the principal optimization experiment.

TABLE VIII: Paired Comparison: Unoptimized Baseline vs. Optimized-Notebook Comparative Evaluation (7 Architectures Present in Both)

	Unoptimized			Optimized-comparative		
Model	**Acc.**	**F1**	κ	**Acc.**	**F1**	κ
ResNet50	0.7308	0.7155	0.3835	0.7372	0.7158	0.3858
VGG16	0.6266	0.5734	0.0925	0.6442	0.5478	0.0845
VGG19	0.7067	0.6596	0.2802	0.6442	0.5777	0.1155
ResNet101	0.7356	0.7143	0.3826	0.7083	0.6693	0.2946
MobileNetV2	0.7436	0.7263	0.4074	0.7003	0.6449	0.2557
MobileNet	0.6554	0.6110	0.1691	0.6715	0.6035	0.1759
InceptionResNetV2	0.8077	0.7968	0.5596	0.6875	0.6214	0.2145

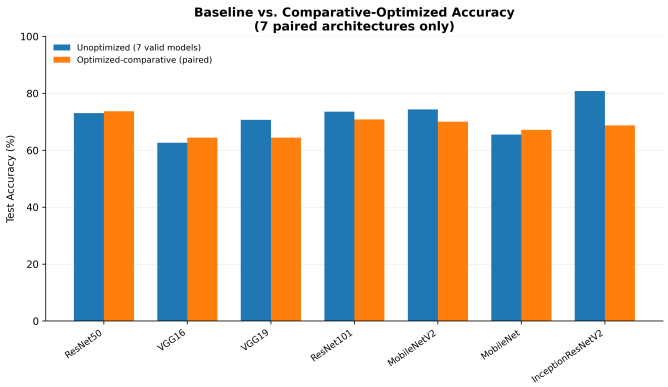

Fig. 7: Paired test accuracy for the seven architectures with verified results in both comparative runs.

VI. Discussion

The central finding of this study is that targeted fine-tuning of InceptionResNetV2 – unfreezing its final 80 layers in a second training stage and extracting a globally-pooled 1,536-dimensional feature representation – substantially improved test performance over the same architecture used as a frozen feature extractor with an untrained classification head, raising accuracy from 0.6875 to 0.8606, weighted F1 from 0.6214 to 0.8523, and Cohen's kappa from 0.2145 to 0.6799. Because this comparison holds the backbone architecture fixed and varies only the training regime, it provides more direct evidence for the value of fine-tuning than any comparison across different architectures could. This pattern is broadly consistent with Choudhury's [3] finding that fine-tuned backbones outperformed frozen-backbone configurations by an average of 5.5 percentage points on a comparable pediatric pneumonia task, though the present study's improvement is considerably larger in absolute terms; this difference in magnitude is plausibly attributable to the fact that the frozen InceptionResNetV2 configuration in this study used an untrained, randomly initialized classification head, whereas frozen-backbone configurations in the literature more commonly retain a task-relevant pretrained head.

The per-class results in Section VD show that fine-tuning improved NORMAL-class precision from 0.87 to 0.99 and PNEUMONIA-class recall from 0.95 to 0.99, but NORMAL-class recall remained the weakest metric in the fine-tuned configuration at 0.64. This asymmetry indicates that the fine-tuned model, while substantially improved overall, still misclassifies a meaningful fraction of NORMAL images as PNEUMONIA. Given the class imbalance in the training set (Section IIIB) and the clinical asymmetry between the cost of a missed pneumonia diagnosis and the cost of a false positive, this residual bias toward the PNEUMONIA class is plausible but is not something this study can attribute with confidence to any single cause without further ablation.

It is worth noting explicitly that InceptionV3's accuracy in the eleven-architecture comparative table (0.8606, Table V) is numerically identical to the fine-tuned InceptionResNetV2 result to four decimal places, despite the two being different architectures evaluated under entirely different procedures – one a frozen, untrained-head comparative run, the other a two-stage fine-tuning experiment with a globally-pooled feature representation. Code inspection confirmed that InceptionV3's result is genuinely computed rather than a copy-paste artifact of the kind identified in the unoptimized notebook (Section VII), and the two results are treated as an unrelated numerical coincidence rather than evidence of any relationship between the two models or procedures.

The paired comparison in Section VE shows that re-running the identical, untrained comparative pipeline a second time did not uniformly improve results: only ResNet50 improved on accuracy, F1, and Cohen's kappa simultaneously between the two runs, while VGG19, ResNet101, MobileNetV2, and InceptionResNetV2 declined on all three metrics simultaneously, and VGG16 and MobileNet show mixed results across metrics. Because both runs use the same unmodified pipeline with no intentional architectural or training change, this variation is attributed to run-to-run variability inherent in training an untrained classification head from a different random initialization, rather than to any deliberate optimization. This finding is not something the reviewed literature models stylistically, since none of the five reviewed studies report a comparative configuration performing worse after a nominally "improved" procedure [1]–[5]; it is reported here because an accurate account of the comparative pipeline's behavior is judged more valuable than omitting an inconvenient result. This finding also underscores why the same-backbone frozen-versus-fine-tuned comparison, rather than any cross-run or cross-architecture comparison, is treated as the central optimization result of this paper.

Finally, this study's absolute accuracy figures are markedly lower than several of the reviewed papers – for example, Chedsom's [1] 97.83% with a trained VGG16 head, or Choudhury's [3] 99.43% with fine-tuned ResNet50. This gap is not interpreted as evidence that the present study's fine-tuning procedure is less effective; rather, it reflects concrete methodological differences, including the comparative pipeline's use of an untrained classification head (Section IVB), the absence of architecture-specific pixel normalization in that same pipeline (Section IIID), and dataset/split differences relative to the reviewed studies (Section VI, above). No claim of state-of-the-art or best-in-class performance is made anywhere in this paper.

VII. Limitations and Reproducibility Considerations

This study's preparation involved a forensic review of both notebooks' executed code and printed output, undertaken specifically to ensure that every reported result in this paper is directly traceable to code that genuinely produced it. This review surfaced several issues that are documented here rather than hidden, since an honest account of these issues is judged to strengthen rather than weaken the paper's academic defensibility.

Mislabeled architecture sections in the unoptimized notebook. Four of the eleven nominal architecture sections in the unoptimized notebook – labeled InceptionV3, DenseNet169, DenseNet121, and XceptionNet – did not instantiate their labeled backbone when the underlying code was inspected directly; the section labeled InceptionV3 in fact instantiated MobileNet, and the sections labeled DenseNet169, DenseNet121, and XceptionNet all instantiated InceptionResNetV2. An additional issue compounded this for the XceptionNet section specifically, whose result variables were overwritten by a naming collision with the preceding DenseNet121 section. Rather than reporting these four results under their nominal (incorrect) labels, or silently substituting the architecture that was actually run, this paper omits all four from the reproducible baseline reported in Table IV and reports only the seven architectures whose code was verified to match their label. This is treated as a data-integrity requirement rather than a stylistic choice: reporting a result under the wrong architecture's name would misattribute that result in a way a reader could not detect without independently re-running the code.

No confusion matrix. Exact prediction arrays (y_{true}, y_{pred}) were not available for direct reconstruction of a confusion matrix for any model reported in this paper. Rather than back-calculating an approximate confusion matrix from rounded precision, recall, and support values – which would produce integer cell counts that could not be verified against the notebooks' actual output and could visually overstate the precision of the underlying data – this paper omits the confusion matrix entirely and instead reports the per-class precision, recall, F1 score, and support values exactly as printed by the notebooks' own classification-report output (Table VII, Section VD).

Dataset count discrepancy. The Chest X-Ray Pneumonia dataset is commonly cited in the literature as containing 5,863 images [2]; this study's data loader, however, processed 5,840 images. Part of this discrepancy is explained by the loader's behavior: the dataset's shipped validation folder contained only 32 images at load time, below a minimum-validation-size threshold built into the loader, which triggered a re-partitioning of the validation split out of the training folder.

This mechanism does not, however, fully account for the exact 23-image gap between the commonly cited total and the total actually processed; a possible contributor is a small number of images that failed to decode during loading, but this was not separately logged in the executed pipeline and is not claimed as a confirmed explanation. The 5,840-image count is reported throughout this paper as the directly observed, authoritative figure for this study's actual experimental conditions.

Derived class-distribution estimate. The exact per-class breakdown of the training set was not printed directly by the data loader. The approximate distribution reported in Section IIIB (1,140 NORMAL, 3,293 PNEUMONIA) is derived by inverting the balanced class weights computed for the fine-tuning experiment against the known training-set size, and is reported as a derived estimate rather than a directly measured count.

Untrained comparative classification head. The common comparative feature-extraction pipeline (Section IVB) uses a randomly initialized Flatten-based classification head that is trained for only 10 epochs with no class weighting, in contrast to the fine-tuning pipeline's more extensively regularized and class-weighted classifier. This asymmetry is a genuine feature of the two notebooks' design, not an omission introduced in the writing of this paper, and it likely contributes to the large absolute performance gap between the comparative and fine-tuned results reported in Section V. It also means that the eleven-architecture comparative results in Table V should not be read as each architecture's best achievable performance on this task, only as a relative ranking under one specific, deliberately lightweight comparative procedure.

Hand-chosen classifier hyperparameters. The classical classifiers evaluated in both the comparative and fine-tuning pipelines (KNN, SVM, random forest, AdaBoost, and XGBoost) used hand-chosen hyperparameters rather than hyperparameters selected via a systematic search procedure such as grid search or Bayesian optimization. Reported classifier performance in this paper should be interpreted as a single configuration's result rather than each classifier's optimal achievable performance.

Single train/validation/test split. All results in this paper are reported on a single fixed split of the dataset (Table I), with no cross-validation performed. Reported metrics should therefore be interpreted with the understanding that no variance estimate across multiple splits is available.

Literature comparability. The five reviewed related-work papers (Section II) differ from this study in dataset composition, size, and split ratio, and in at least one case (Satish Chandra et al. [4]) use an entirely different dataset drawn from a different patient population. Numerical comparisons to these papers, where made in Section VI, are offered only as qualitative context and are not intended to imply direct, controlled comparability. One reviewed source [2] was published on a journal website found to contain unrelated spam-adjacent content alongside its articles, a soft signal of publishing-venue quality noted here for transparency rather than resolved.

VIII. Conclusion

This paper reported a two-part study of pneumonia classification on the Chest X-Ray Pneumonia dataset. First, a reproducible seven-architecture baseline comparison and an independent eleven-architecture comparative evaluation were conducted using a common, untrained feature-extraction pipeline, from which InceptionResNetV2 was identified as the strongest candidate architecture in the baseline comparison (accuracy 0.8077). Second, a targeted fine-tuning experiment was applied specifically to InceptionResNetV2, using a two-stage training schedule and a globally-pooled 1,536-dimensional feature representation, achieving a substantially improved test accuracy of 0.8606, weighted F1 of 0.8523, and Cohen's kappa of 0.6799 relative to the same architecture's frozen, untrained-head configuration (accuracy 0.6875, F1 0.6214, κ 0.2145). This same-backbone, frozen-versus-fine-tuned comparison – rather than any comparison across different architectures – is the central evidence this paper offers for the value of targeted fine-tuning on this task.

A forensic review of both notebooks' executed code, undertaken during this paper's preparation, identified that four of the eleven nominal architecture sections in the original notebook did not correctly instantiate their labeled backbone; these results were excluded from the reproducible baseline rather than reported under an incorrect label, and this exclusion is documented in Section VII alongside several other reproducibility considerations. This paper does not claim state-of-the-art performance, and its absolute accuracy figures should be read in the context of the comparative pipeline's untrained classification head and the dataset/preprocessing differences from the reviewed literature discussed in Section VI.

Future work could extend the targeted fine-tuning procedure applied here to InceptionResNetV2 to the remaining ten comparative architectures under a matched, controlled protocol, enabling a genuine cross-architecture fine-tuning comparison rather than the single-architecture case reported here. Systematic hyperparameter search for the classical classifiers, a class-weighted and more extensively trained comparative classification head, and Grad-CAM-based qualitative analysis of the fine-tuned model's misclassifications – particularly its residual NORMAL-class recall of 0.64 – are also natural directions for continued work.

References

[1] P. Chedsom, "Pneumonia detection from chest x-ray images using convolutional neural networks and transfer learning techniques," *Journal of Applied Informatics and Technology*, vol. 7, no. 2, pp. 406–431, 2025.

[2] G. C. A. Wibowo, D. M. Simanjuntak, and H. J. Christanto, "Comparison of cnn, resnet50, and vgg16 for pneumonia classification using transfer learning," *International Journal of Health Engineering and Technology*, vol. 5, no. 2, 2026.

[3] A. R. Choudhury, "Pediatric pneumonia detection from chest x-rays: A comparative study of transfer learning and custom CNNs," *arXiv preprint arXiv:2601.00837*, 2025.

[4] N. Satish Chandra, S. Raj, and B. S. Mahanand, "NGBoost classifier using deep features for pneumonia chest x-ray classification," *Applied Sciences*, vol. 15, no. 17, p. 9821, 2025.

[5] I. Pal and A. Pal, "A hybrid deep learning approach for pneumonia detection from chest x-ray images using custom CNN and transfer learning," *INFOCOMP Journal of Computer Science*, vol. 24, no. 2, 2025.